CEUB

EDUCAÇÃO SUPERIOR

Segurança Digital

Prof. Henrique Evaristo

ceub.br

Segurança Digital

Introdução ao Linux

Laboratório – Antes de colocar a mão na massa...

Introdução aos Conceitos de Criptografia

A criptografia é a ***ciência de transformar dados em um formato ininteligível para protegê-los contra acessos indevidos.*** Para a nossa aula, é importante que os alunos compreendam que existem abordagens diferentes para problemas diferentes:

Criptografia Simétrica: Usa a *mesma chave* tanto para embaralhar (criptografar) quanto para desembaralhar (descriptografar) os dados. É extremamente rápida e usada para proteger grandes volumes de dados (ex: criptografia de um disco rígido ou de um banco de dados de clientes).

Criptografia Assimétrica: Usa um *par de chaves* matematicamente relacionadas (uma chave pública, que todos conhecem, e uma chave privada, mantida em segredo). É a base da segurança da internet (como os certificados TLS/SSL) e resolve o problema de como distribuir chaves de forma segura.

Laboratório – Antes de colocar a mão na massa...

Porém, existe uma terceira categoria na família criptográfica que não serve para "esconder" a mensagem, mas sim para **provar que ela é autêntica:** **As Funções de Hash**.

O Conceito de Hash (Função de Resumo Criptográfico)

Diferente da **criptografia tradicional**, que **é um processo de "ida e volta"** (você tranca o dado e depois destranca para ler), o **Hash é um processo unidirecional (apenas ida)**.

Laboratório – Antes de colocar a mão na massa...

O que é HASH

Uma função de **hash é um algoritmo matemático que pega uma entrada de dados** de qualquer tamanho (pode ser uma simples senha de 8 caracteres, um arquivo de texto, ou um banco de dados inteiro de transações financeiras) **e a transforma em uma saída de tamanho fixo** que pode ser **chamada de *hash value*, *message digest* ou simplesmente *hash*.**

Laboratório – Antes de colocar a mão na massa...

Características fundamentais de um bom algoritmo de Hash

Unidirecionalidade (One-Way): É computacionalmente inviável pegar o resultado do hash e fazer a engenharia reversa para descobrir qual era a mensagem original.

Tamanho Fixo da Saída: Não importa se você processou a palavra "Olá" ou a enciclopédia inteira; se o algoritmo (como o SHA-256) gera uma saída de 256 bits, o hash final terá sempre 256 bits.

Efeito Avalanche: Se você alterar um único bit na entrada (exemplo: mudar um valor de transferência bancária de R$ 1.000,00 para R$ 1.000,01), o hash resultante mudará de forma drástica e imprevisível.

Resistência à Colisão: Deve ser estatisticamente quase impossível encontrar duas entradas de dados diferentes que gerem exatamente o mesmo hash de saída. (Algoritmos antigos, como o MD5, caíram em desuso justamente porque hackers conseguiram gerar colisões).

Laboratório – Antes de colocar a mão na massa...

Aplicações Práticas do Hash (Protegendo a Integridade)

Se o hash não permite ler a mensagem original, para que ele serve?

Ele é o principal controle técnico para garantir a **Integridade** e a **Autenticidade**.

Armazenamento Seguro de Senhas: Sistemas modernos (e instituições financeiras) nunca armazenam a senha do usuário em texto claro no banco de dados. Eles armazenam apenas o *hash* da senha. Quando o usuário faz login, o sistema calcula o hash da senha digitada e compara com o hash salvo. Se forem iguais, a senha está correta. Se o banco de dados for vazado, os atacantes roubarão apenas hashes ininteligíveis, não as senhas reais.

Laboratório – Antes de colocar a mão na massa...

Aplicações Práticas do Hash (Protegendo a Integridade)

Se o hash não permite ler a mensagem original, para que ele serve?

Ele é o principal controle técnico para garantir a **Integridade** e a **Autenticidade**.

Assinaturas Digitais: Quando um documento legal ou um script de alteração de infraestrutura é assinado digitalmente, o que é assinado, na verdade, é o hash do documento. Isso garante que, se uma única vírgula for alterada após a assinatura, o hash não baterá mais, invalidando a assinatura.

Verificação de Integridade de Arquivos: Ao transferir grandes volumes de dados (como pacotes de atualização de sistemas ou lotes de transações financeiras), o sistema emissor envia os dados acompanhados do seu hash. O sistema receptor recebe os dados, recalcula o hash e compara. Se os hashes forem idênticos, há garantia matemática de que o arquivo não corrompeu na rede e não foi adulterado no meio do caminho (ataque *Man-in-the-Middle*)

Laboratório – Antes de colocar a mão na massa...

Evolução e Padrões Atuais de Algoritmos de Hash

Para aplicar a criptografia no dia a dia, o mercado de tecnologia padronizou diversas famílias de algoritmos de hash ao longo das décadas. É crucial entender quais são seguros para uso atual e quais servem apenas como exemplo do que **não** fazer.

Os Obsoletos (Não recomendados para novos projetos)

- **MD5 (Message Digest 5)** Criado no início dos anos 90, gera um hash de 128 bits. Hoje, é considerado criptograficamente quebrado. Com o poder computacional atual, é trivial gerar "colisões" (dois arquivos diferentes gerando o mesmo hash MD5). Só deve ser usado para verificações simples de erro de download, nunca para segurança.

- **SHA-1 (Secure Hash Algorithm 1)** Gera hashes de 160 bits. Foi o padrão da internet por muito tempo, mas em 2017 a Google provou na prática a sua quebra (o ataque *SHAttered*). Foi totalmente aposentado por navegadores e indústrias críticas.

Laboratório – Antes de colocar a mão na massa...

Os Padrões de Mercado (Para integridade de arquivos e documentos)

- **Família SHA-2 (SHA-256 e SHA-512)** É o atual "cavalo de batalha" da internet. O SHA-256 (que gera uma saída de 256 bits) é extremamente seguro e amplamente utilizado em certificados digitais (HTTPS/TLS), assinaturas de documentos com validade jurídica e até mesmo na rede Bitcoin. É o requisito mínimo em auditorias rigorosas, como as do Sistema Financeiro.

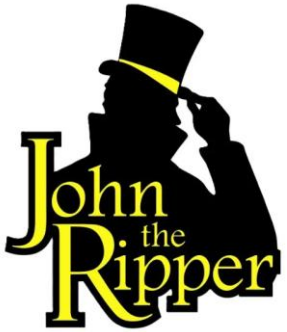
- **Família SHA-3 (Keccak)** É o padrão mais recente aprovado pelo NIST (Instituto Nacional de Padrões e Tecnologia dos EUA). Ele possui uma estrutura matemática completamente diferente do SHA-2, servindo como uma alternativa ultra-segura caso alguma vulnerabilidade futura seja descoberta na família anterior.

Laboratório – Kali Linux

Laboratório – Quebra de Senha com a **ferramenta john**

Este laboratório será realizado em máquina virtual utilizando o Kali Linux, seu objetivo é mostrar como senhas de arquivos podem ser quebradas.

Laboratório – Kali Linux



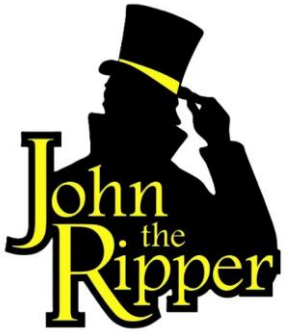
John the Ripper quebra a senha do PDF

A Fase de Extração (Separando o alvo do arquivo) O JtR não trabalha diretamente com o arquivo .pdf. O primeiro passo da ferramenta é extrair o *hash* ou os dados de validação da criptografia do documento.

A Fase de Ataque (O Motor de Força Bruta / Dicionário) Com o hash extraído e a lista de senhas (wordlist) criada por vocês em mãos, o JtR entra em ação. O funcionamento interno dele segue este loop incansável:

- **Leitura:** O John lê a primeira palavra da wordlist criada em sala.
- **Transformação:** Ele pega essa palavra e a passa pelo *mesmo* algoritmo de criptografia/hash que o PDF usou (por exemplo, derivando uma chave RC4 ou AES usando a palavra como semente).
- **Comparação (A prova real):** Ele compara o resultado matemático que acabou de gerar com o hash que extraímos no Passo 1.
  - Se for **diferente**, a senha está errada. Ele descarta e passa para a próxima palavra da lista em milissegundos.
  - Se for **igual**, ocorre uma colisão intencional. O John the Ripper para o processo e avisa: *"A palavra que gerou esse hash é a senha correta!"*

Laboratório – Kali Linux



John the Ripper quebra a senha do PDF

A Fase de Extração (Separando o alvo do arquivo) O JtR não trabalha diretamente com o arquivo .pdf. O primeiro passo da ferramenta é extrair o *hash* ou os dados de validação da criptografia do documento.

A Fase de Ataque (O Motor de Força Bruta / Dicionário) Com o hash extraído e a lista de senhas (wordlist) criada por vocês em mãos, o JtR entra em ação. O funcionamento interno dele segue este loop incansável:

- **Leitura:** O John lê a primeira palavra da wordlist criada em sala.
- **Transformação:** Ele pega essa palavra e a passa pelo *mesmo* algoritmo de criptografia/hash que o PDF usou (por exemplo, derivando uma chave RC4 ou AES usando a palavra como semente).
- **Comparação (A prova real):** Ele compara o resultado matemático que acabou de gerar com o hash que extraímos no Passo 1.
  - Se for **diferente**, a senha está errada. Ele descarta e passa para a próxima palavra da lista em milissegundos.
  - Se for **igual**, ocorre uma colisão intencional. O John the Ripper para o processo e avisa: *"A palavra que gerou esse hash é a senha correta!"*

Laboratório – Kali Linux

 # extrair hash

```
pdf2john arquivoprotegido.pdf > protegido.hash
```

 # atacar com John usando rockyou

```
john --wordlist=/usr/share/wordlists/rockyou.txt protegido.hash
```

 # conferir a senha (mas pode ser um cat normal)

```
john --show protegido.hash
```

Laboratório – E a Proteção ? ? ?

Hashes Específicos para Armazenamento de Senhas (O foco do nosso laboratório)

Atenção: algoritmos como o SHA-256 são projetados para serem extremamente rápidos. Isso é ótimo para validar um arquivo de 10 GB, mas é péssimo para guardar senhas. Se um hash é rápido de calcular, ferramentas de força bruta (como o John the Ripper) conseguem testar bilhões de senhas por segundo.

Para **senhas, usamos algoritmos propositalmente lentos** e que exigem muito processamento e memória (técnica chamada de *Key Stretching*), além de adicionarem um valor aleatório (o *Salt*) para impedir ataques de dicionário pré-computados:

- **Bcrypt:** O padrão de fato há muitos anos. Ele é configurável, permitindo que você aumente o "custo" de processamento conforme os computadores ficam mais rápidos, atrasando ataques de força bruta.
- **Argon2:** O vencedor da *Password Hashing Competition*. É o estado da arte atual para armazenamento de senhas, pois além de exigir tempo de CPU, ele exige alto consumo de memória RAM, tornando ataques usando placas de vídeo (GPUs) inviáveis financeiramente para o atacante.

Obrigado

Prof. Henrique Evaristo

CEUB

EDUCAÇÃO SUPERIOR